JAVA OOP Laboratory Exercise: Campus Café_Ordering System

ITS 202: Objected Oriented Programming

Submitted by:

Calibgos, Julie Ann A.

Tag, Angel Rose

Section C

Nov. 10, 2026

Abstract

We used abstraction in the MenuItem class by making it an abstract class and declaring computePrice() as an abstract method. This allows each menu item, such as Beverage, Pastry, and Meal, to have its own way of calculating the price.

Encapsulation

We used encapsulation by making attributes private, such as itemCode, size, warmed, and loyaltyPoints. We used getters and setters to access and update their values safely.

Inheritance

We used inheritance when Beverage, Pastry, and Meal extended the MenuItem class. This allows the child classes to reuse the attributes and methods of the parent class.

Method Overriding

We used method overriding when the Beverage, Pastry, and Meal classes provided their own versions of computePrice() and getDescription(). This makes each menu item display its own description and calculate its price based on its features.

Constructor Overloading

Base of the instruction, we used constructor overloading in the Customer class by creating three constructors with different parameters: Customer(), Customer(String name), and Customer(String name, boolean student). This allows me to create customers with different information.

Super

We used the super keyword in the Beverage, Pastry, and Meal classes to call the constructor and methods of the parent class, MenuItem. This helps avoid repeating the same code.

If Statement

I used if statements to check conditions in the program. For example, in the Beverage class, I checked the size of the drink to determine the additional price. I also used it to validate prices and customer information.

If-Else Statement

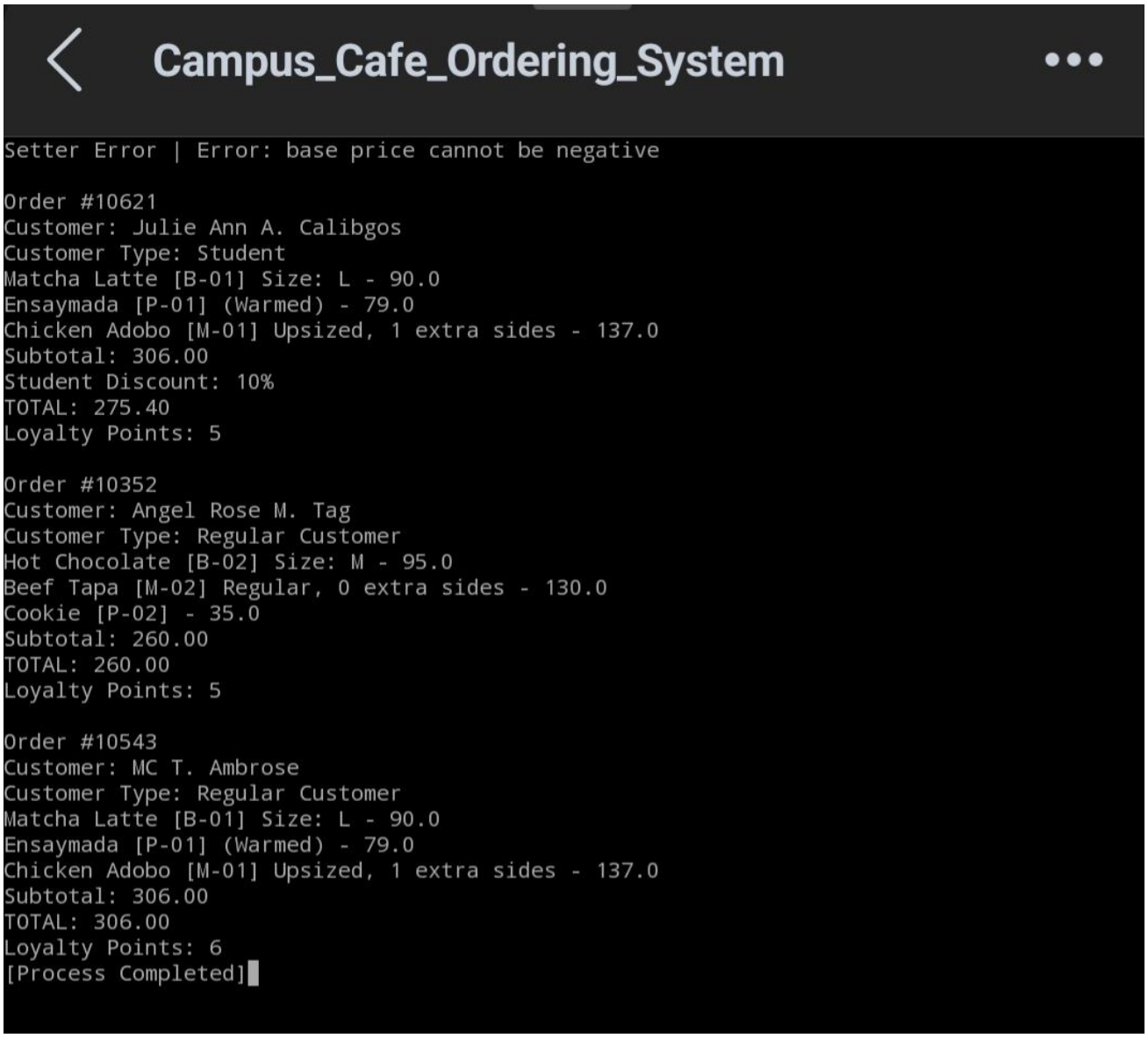
I used if-else statements to make decisions based on certain conditions. For example, in the Order class, the program checks whether the customer is a student or a regular customer to determine if they will receive a 10% discount.

Classes and Objects

We have seven classes to organize the Café Ordering System, ***MenuItem*** that serves as the parent class for all menu items. ***Beverage*** it represents drinks with different sizes. ***Pastry*** represents

pastries that can be warmed. ***Meal*** represents meals with optional upgrades and extra sides. ***Customer*** it stores customer information and loyalty points. ***Order*** manages orders, prices, discounts, and vouchers. ***CafeApp*** contains the main method that runs the program.

Screenshots of the Program's Actual Output



Challenges that we encountered and how we solved them.

#Applying If and If-Else Statements

We found it challenging to create conditions for checking beverage sizes, pastry warming, meal upgrades, and student discounts. But we used if and if-else statements to check the conditions and perform the correct actions based on the given values.

#Creating and Connecting Seven Classes

We had difficulty understanding how to connect the seven classes and make them work together in one program because during our discussions we usually used 3-5 classes, so we do is organized the program into MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp, then connected them using objects and inheritance.

#Calculating Discounts and Vouchers

We found it challenging to calculate the student discount and subtract voucher amounts without making the total negative. So we used conditional statements to apply the 10% student discount and added a condition that prevents the final total from going below zero.

#Using the Abstract Class (MenuItem)

We found it difficult to understand why I could not create a direct object of the abstract MenuItem class. We used MenuItem as a parent class and created objects from its child classes, such as Beverage, Pastry, and Meal.

#Validating Beverage Sizes

We had difficulty making sure that customers could only select Small, Medium, or Large sizes. But we used an if statement in setSize() to accept only “S”, “M”, or “L” and display an error for invalid sizes.

#Handling Empty Menu Items

We needed to make sure the program would not calculate the price of an item that had not been added to the order.We used if (item1 != null), if (item2 != null), and if (item3 != null) before calculating the subtotal.